HopIn-University Carpooling Platform

Team Members: Jessica, Rida, Rudra, Suhayb, Youssef

Introduction

This project presents **HopIn**, a web-based carpooling platform designed specifically for students and staff at the University of Regina. The goal of HopIn is to provide a safer, more affordable, and more organized way for users to share rides to and from campus.

Commuting to university can be challenging due to rising transportation costs, limited public transit options, and lack of coordination between individuals traveling similar routes. HopIn addresses these challenges by connecting drivers with available seats to riders seeking reliable transportation within a trusted university-only environment.

The system was designed and developed following key software engineering principles, including **Model-View-Controller (MVC) architecture**, **domain-driven design**, and an **Agile/Scrum-inspired process**. This report outlines the team's exploration, design decisions, development process, and final outcomes.

Project Goals

The primary goal of this project was to design and develop a functional Minimum Viable Product (MVP) of a carpooling platform that demonstrates core ride-sharing functionality.

The key objectives included:

- Developing a web-based application using MVC architecture
- Designing a clear and structured domain model
- Implementing essential ride-sharing features (posting, searching, requesting rides)
- Creating an intuitive and user-friendly interface
- Demonstrating effective teamwork and Agile project management

Additionally, the team aimed to build a system that solves a real-world problem while remaining feasible within a two-month development timeline.

Process & Exploration

At the beginning of the project, the team explored multiple potential ideas, including a skill-sharing platform and a scheduling system. While these ideas were viable, they lacked a strong real-world connection and clear user interaction flows.

The team ultimately selected a **carpooling platform** because:

- It solves a common and relatable problem among students
- It has a well-defined domain (users, rides, requests)
- It fits naturally within MVC architecture
- It allows for meaningful user interaction

Throughout the planning phase, the team focused on defining a **realistic MVP scope**. Early ideas included features such as messaging, payments, and real-time tracking, but these were intentionally excluded to keep the project manageable.

The development process followed an Agile-inspired approach, where tasks were broken into smaller units and tracked using a **Kanban board**. This allowed the team to monitor progress and adapt as needed.

System Design

1 Architecture

The HopIn system follows the **Model-View-Controller (MVC)** architecture:

- **Model:** Handles data and database interactions (users, rides, requests, ratings)
- **View:** Represents the user interface (forms, dashboards, listings)

- **Controller:** Manages logic, processes user actions, and connects the model to the view

This separation of concerns improves maintainability, scalability, and clarity of the system.

2 Domain Model

The system is built around the following key entities:

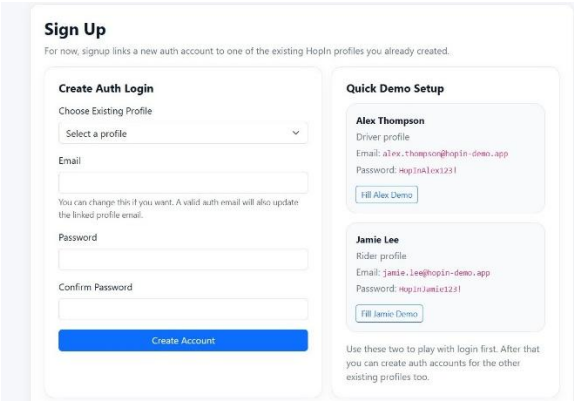
- **User:** Represents a student or staff member using the platform
- **Ride:** Represents a trip offered by a driver
- **RideRequest:** Represents a request made by a rider to join a ride
- **Rating:** Represents feedback provided after a completed ride
- **Message:** Enables communication between users

These entities form the foundation of the system and enable structured interactions between users.

MVP Features & Implementation

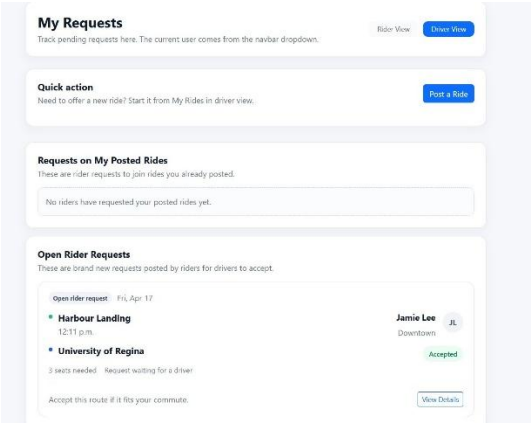
The team successfully implemented the following core features:

1. **User Authentication**
   Users can create accounts and log in securely. This ensures that only registered users can access the platform and interact with ride-sharing features.



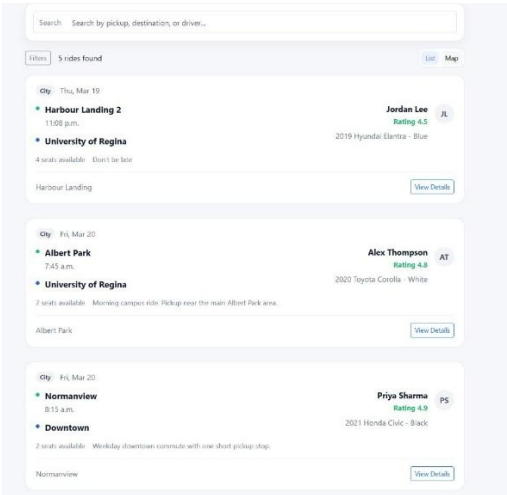
2. **Driver Ride Posting**
   Drivers can create ride listings by providing details such as destination, date, time, number of available seats, and pickup location.

   This feature enables drivers to offer rides to other users.

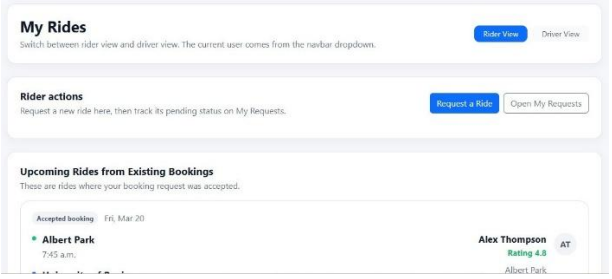


3. **Ride Browsing and Searching**
   Riders can browse available rides and filter them based on criteria such as time, location, and availability.

   This allows users to quickly find rides that match their needs.

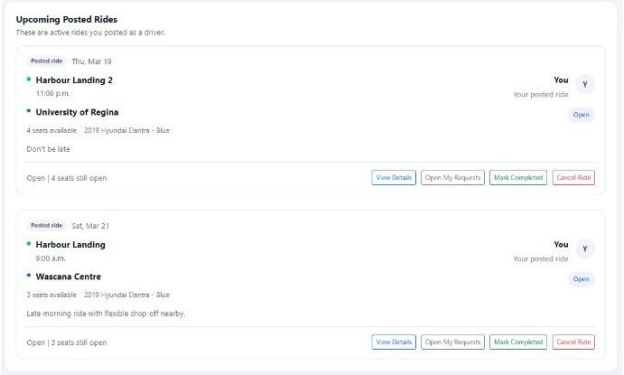


4. **Ride Request System**
   Riders can send requests to join a ride, and drivers can review and respond to these requests.
   This ensures controlled access to rides and prevents overbooking.

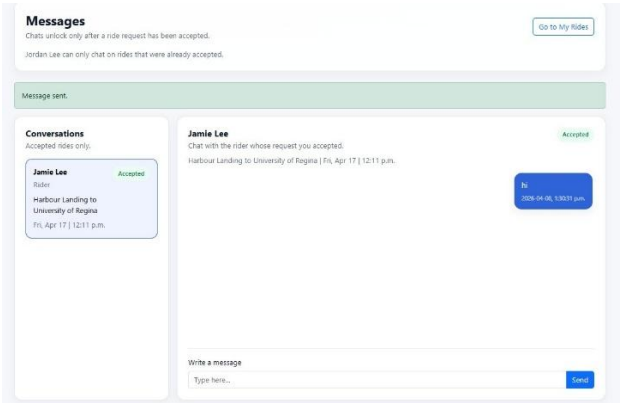


5. **Ride Confirmation**
   Drivers can accept or reject ride requests. Once accepted, the ride is confirmed and both users can proceed with the trip.

   This feature completes the core ride-sharing workflow.



6. **Messaging**
   A basic messaging system allows drivers and riders to communicate regarding ride details, improving coordination and clarity.



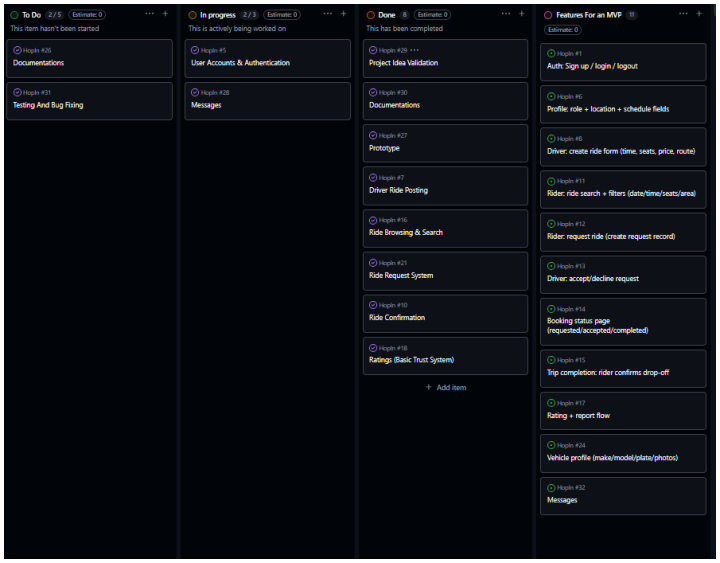
Project Management & Kanban Workflow

The team used a Kanban-based workflow to manage the development of the HopIn platform. Tasks were organized into three columns: To Do, In Progress, and Done, allowing the team to clearly track progress throughout the project.

Tasks included both project management activities (such as documentation and planning) and development tasks (such as implementing core features). Each task was assigned to team members to ensure accountability and balanced workload.

As the project progressed, tasks were moved across the board, providing a clear visual overview of the team's progress and helping identify any delays or issues. This approach supported an iterative development process, where the team focused first on core MVP features before refining and testing the system.

Overall, the Kanban workflow improved organization, communication, and efficiency, contributing to the successful completion of the project.



Feedback & Iteration

Throughout the project, the team received feedback from the instructor and peers. One key piece of feedback was to **limit the scope of the MVP** to ensure successful completion within the given timeframe.

As a result:

- Advanced features such as real-time tracking and payments were removed
- The focus shifted to core ride-sharing functionality
- The user interface was simplified to improve usability

Additionally, feedback on the prototype led to improvements in layout and navigation, making the system more intuitive for users.

Testing & Validation

The team conducted testing on all major features of the application, including:

- User authentication
- Ride posting
- Ride searching
- Ride requests and confirmations

During testing, several issues were identified, such as:

- Incorrect request status updates
- Minor UI inconsistencies
- Data handling errors

These issues were resolved through debugging and iterative improvements. Testing ensured that the system was functional, stable, and ready for demonstration.

Reflection & Lessons Learned

This project provided valuable insights into both technical development and teamwork.

From an individual perspective, team members learned the importance of balancing attention to detail with overall progress. While striving for high-quality work is important, focusing too much on minor details can slow down development.

From a team perspective, communication was a key factor in success. The team learned that:

- Clear task distribution improves efficiency
- Regular check-ins help maintain alignment
- Flexibility is necessary when challenges arise

One challenge faced was coordinating different parts of the system, particularly integrating frontend and backend components. This was overcome through collaboration and incremental testing.

The project also reinforced the importance of **Agile practices**, as breaking work into manageable tasks made the development process more organized and less overwhelming.

Overall, the experience highlighted that successful software development is not just about coding, but also about planning, communication, and adaptability.

Conclusion

The HopIn project successfully achieved its goal of developing a functional MVP for a university carpooling platform. The system demonstrates key software engineering principles, including MVC architecture, domain-driven design, and Agile project management.

The final product provides a practical solution to a real-world problem, offering students and staff a safer and more efficient way to share rides.

While there are opportunities for future improvements, such as adding real-time features and enhanced security, the current system provides a strong foundation for further development.